More korololeg@gmail.com Dashboard Sign Out

ВОСКРЕСЕНЬЕ, 29 АВГУСТА 2010 Г.

Длинное DIV длинное & Длинное MOD длинное

[Вся длинная арифметика]

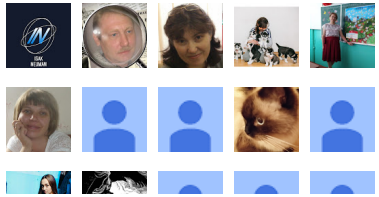
Пришло время познакомиться с самой “страшной” операцией на длинную арифметику. Первый раз я ее реализовал на **Accepted** 31 июля 2008. До этого момента мне было даже страшно подумать над ее реализацией. Но как оказалось не так уж это деление и страшно, как его малюют. Мы не будем изобретать велосипед, а просто грамотно, максимально понятно и быстро реализуем деление столбиком. Рассмотрим нижепредставленный рисунок.

Как видно osn = 100.

Заметим факт, что количество разрядов у частного от деления не превосходит количества разрядов у делимого. Т.е. начинаем формировать ответ со старшего разряда. На каждой итерации

ПОСТОЯННЫЕ ЧИТАТЕЛИ

Followers (100) Next



ПОИСК ПО ЭТОМУ БЛОГУ

Поиск

АВТОРЫ

Danil

slipstak2

имеем текущее значение, которое пытаемся уменьшить на максимально большое количество раз делимым. Итак: как будем искать это “максимально большое количество”? За ответом отсылаю в нашу любимую первую главу **“Программирование в алгоритмах”**. Ответ прост. Вместо того, чтобы линейно перебирать цифры в интервале от 0 до osn в поисках нужного ответа, мы будем использовать бинарный поиск.

Корректность этого утверждения вытекает из того, что рассматриваемая функция, которую можно представить в виде b * x(где b – делимое, x – подбираемое значение) является монотонной. В нашем случае возрастающей.

Оценим **выигрыш** в скорости относительно “лобовой реализации”. Для osn = 10000 предложенная реализация будет работать на 10000/log(10000) ~ 769 раз быстрее.

Рассмотрим конкретную реализацию целочисленного деления двух длинных чисел:

```
void LevelUp()
{
  for (int i = amount;i>=1;i--)
    digits[i] = digits[i-1];
  if (digits[amount])
    amount++;
}
...
BigInt operator / (const BigInt &a, const BigInt &b)
{
  BigInt res;
  BigInt curValue;
  for (int i = a.amount-1; i>=0; i--)
```

СЧЕТЧИК ПОСЕТИТЕЛЕЙ

Visitors See more
148,498 1,276 529
38,240 1,223 502
16,132 1,084 488
7,833 1,055 453
5,656 960 433
1,377 737 399
Pageviews: 497,741
FLAG counter

АРХИВ БЛОГА

► 2015 (1)
► 2013 (1)
► 2012 (18)
► 2011 (50)
▼ 2010 (64)
► декабря (7)
► ноября (9)
► октября (2)
► сентября (14)
▼ августа (11)
Длинное^короткое (возведение в степень)
Длинное DIV длинное & Длинное MOD длинное

```
14.   {
15.     curValue.LevelUp(); // * osn
16.     curValue.digits[0] = a.digits[i];
17.     // подбираем максимальное число x, такое что b * x <= curValue
18.     int x = 0;
19.     int l = 0, r = osn;
20.     while (l <= r)
21.     {
22.       int m = (l + r) >> 1;
23.       BigInt cur = b * m;
24.       if (cur <= curValue)
25.       {
26.         x = m;
27.         l = m+1;
28.       }
29.       else
30.         r = m-1;
31.     }
32.     res.digits[i] = x;
33.     curValue = curValue - b * x;
34.   }
35.   // избавляемся от лидирующих нулей
36.   int pos = a.amount;
37.   while (pos>=0 && !res.digits[pos])
38.     pos--;
39.   res.amount = pos+1;
40.
41.   return res;
42. }
```

* This source code was highlighted with Source Code Highlighter.

Обратите внимание на строчку 20. Строгая проверка дает неправильное решение. Советую разобраться почему.

Когда мы рассматривали деление длинного на короткое мы столкнулись с тем, что реализация операции взятие остатка от

Длинное DIV короткое & Длинное MOD короткое
Длинное * длинное
Длинное * короткое
Длинное – длинное(знаковое вычитание)
Длинное – длинное (беззнаковое вычитание)
Сравнение длинных чисел
Длинное + короткое
Вывод длинных чисел
Ввод длинных чисел
► июля (2)
► мая (7)
► апреля (1)
► марта (3)
► января (8)
► 2009 (8)

LIVEINTERNET

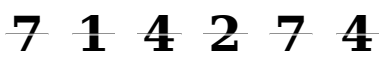
деления появилась путем нехитрой модификации операции целочисленного деления. Как несложно догадаться, здесь мы наблюдаем аналогичную картину:

```
BigInt operator % (const BigInt &a, const BigInt &b)
{
  BigInt res;
  BigInt curValue;
  for (int i = a.amount-1; i>=0; i--)
  {
    curValue.LevelUp(); // * osn
    curValue.digits[0] = a.digits[i];
    // подбираем максимальное число x, такое что b * x <= curValue
    int x = 0;
    int l = 0, r = osn;
    while (l <= r)
    {
      int m = (l + r) >> 1;
      BigInt cur = b * m;
      if (cur <= curValue)
      {
        x = m;
        l = m+1;
      }
      else
        r = m-1;
    }
    res.digits[i] = x;
    curValue = curValue - b * x;
  }
  return curValue;
}
```

* This source code was highlighted with Source Code Highlighter.

АВТОР: SLIPSTAK2 НА 23:14
ЯРЛЫКИ: ДЛИННЫЕ ЧИСЛА

PROJECTEULER



LATEX

29 КОММЕНТАРИЕВ:

Pewk 8 ноября 2010 г., 17:14

ааа можете объяснить строчки 24-30?

Ответить

slipstak2 8 ноября 2010 г., 21:55

в строчках 20-31 происходит поиск числа x, на которое накладываются следующие ограничения:
1) b * x <= curValue
2) Оно должно быть максимальным и удовлетворять условию 1).

Метод, с помощью которого ищем само число называется бинарным. Его реализацию в чистом виде можно посмотреть здесь: http://cppalgo.blogspot.com/2010/07/binarysearch-lowerbound-upperbound.html

Итак, чтобы понять строчки 24-30 предлагаю разобрать пошагово алгоритм на конкретном примере.

На второй итерации алгоритма имеем:
curValue = 1-23, где 1 - старший разряд, 23 - младший
b = 53.

Задача найти такое x, которое:
53 * x <= 123 - неравенство (1)

На каждой итерации вместо числа x будет подставлять переменную m в неравенство (1)

Итерации бинарного поиска:
1) l = 0, r = 100, m = 50 - понятно, что много. При этом нужно сдвинуть правую границу, чтобы сузить границы поиска. Можно просто присвоить r = 50, но мы же уже точно знаем, что 50 - не ответ на наш вопрос, поэтому

можно смело присвоить r = m - 1, не боясь потерять правильный корень.
2) l = 0, r = 49, m = 24
3) l = 0, r = 23, m = 11
4) l = 0, r = 10, m = 5
5) l = 0, r = 4, m = 2 - первый раз выполнится условие cur <= curValue. Это говорит о том, что m - число, которое удовлетворяет неравенству (1), поэтому мы запоминаем это число в итоговую переменную x. Теперь наша задача грамотно подвинуть левую границу. Как и в случае с правой границей мы можем сделать просто l = m. Но мы уже обработали случай с числом m, поэтому можно также смело присвоить l = m + 1. После этой итерации не факт, что найденное число x будет максимальным. Для этого нужно выполнить остальные итерации бинарного поиска.
6) l = 3, r = 4, m = 3.
7) l = 3, r = 2 - конец итераций.

Как можно было заменить "приемчик" с r = m - 1 и l = m + 1 позволяет избежать зацикливания на итерации 6).

Найденное число x = 2. Оно полностью удовлетворяет поставленной задачи.

Ответить

Pewk 9 ноября 2010 г., 20:38

ага, понятно, спасибо огромное

Ответить

Анонимный 11 декабря 2010 г., 14:44

Объясние пожалуйста, как работает функция LevelUp и куда ее нужно вставить в программе

Ответить

slipstak2 12 декабря 2010 г., 14:07

Наверное название функции LevelUp было не очень удачное, т.к. оно не в полной мере отображает ее смысл.

Во-первых, эта функция относится к типу(структуре) BigInt и применяется для объектов этого типа. Это видно в строчке 15 реализации операции "/". Отсюда делаем вывод, что эту функцию нужно поместить внутрь структуры BigInt.

Во-вторых, вполне ризонный вопрос - что делает эта функция? Название функции ответа на этот вопрос не дает. Глядим на ее реализацию. Если все еще остались вопросы. Поясню: эта функция умножает длинное число на основание системы счисления. Допустим мы имеем число 12345 и osn = 100. В памяти мы храним это число в виде массива с первыми треями элементами: 45, 23, 1. После выполнения функции получаем число 1234500, и массив с первыми 4 элементами: 0,45,23,1.
Пример, когда osn = 10 не приводил. Надеюсь читатель это сможет сделать самостоятельно.

Думаю это внесло определенную долю понимания.
Если все же остались вопросы, этот исходник должен их прояснить: http://www.everfall.com/paste/id.php?3jq29e52lbwo

Ответить

Юрий 4 декабря 2011 г., 16:28

Здравствуйте.. смысл деления вроде понятен. Такой вопросик только : почему при вычислениях есть неточность.. например 4200/42 = 99 а при числах побольше результат к примеру 34...04 а правильный 34...4 , т.е. дополнительный 0 в разряде десятков вылезает. Я по-другому немного храню число (с использованием вектора), но код функции не менял. Не подскажете в чем может быть дело? У Вас такого не было? Считает как надо?

Ответить

slipstak2 4 декабря 2011 г., 16:59

1. По поводу 4200/42. У меня получается правильный ответ.
2. Если вылазеет лишний ноль, то думаю что здесь скорее всего проблема с выбором основании системы счисления. Так например, в своем текущем решении у меня выставлены следующие настройки:

```
int len = 4;
const char * digitFormat = "%.4d";
int osn = 1e4;
```

функция output выглядит так:

```
void output()
{
printf("%d",digits[amount-1]);
for (int i= amount-2;i>=0;i--)
printf(digitFormat,digits[i]);
}
```

Следовательно, если будет выводиться цифра числа отличная от старшего разряда, то необходимо предусмотреть возможность вывода лидирующих нулей. Лишний ноль может вылезти, если например osn = 10, а digitFormat = "%.2d";

3. Данную реализацию тестировал здесь:
http://informatics.mccme.ru/moodle/mod/statements/view3.php?id=251&chapterid=141
http://informatics.mccme.ru/moodle/mod/statements/view3.php?id=251&chapterid=142
там проблем не было.

Ответить

Юрий 4 декабря 2011 г., 17:07

не, дело не в выводе.. я смотрю что в массиве хранится и выводится все так как нужно.. так что че-то при выполнении косячит, ток вот где и почему понять не могу.. уже несколько раз прогнал пошагово..

Ответить

slipstak2 4 декабря 2011 г., 17:11

http://www.everfall.com/paste/id.php?3fn39vbuk8aj - вот мое решение

Ответить

Юрий 4 декабря 2011 г., 17:31

Кажется понял в чем дело.. вместо условия (cur <= curValue) я писал (cur < curValue).. ща посмотрим что получится после замены.

Ответить

Анонимный 21 мая 2012 г., 16:46

Спасибо

Ответить

Анонимный 29 августа 2012 г., 12:13

Почини рисунок, будь котиком.

Ответить

slipstak2 29 августа 2012 г., 12:26

Зачем же фамильярничать?

Ответить

Анонимный 10 декабря 2012 г., 12:54

Использовал Ваш % для реализации RSA. Спасибо! Не знаю на сколько существенно, но мне пришлось добавить следующие проверки перед

вычисление:

```
if(a < b){
return a;
}

if(a == b){
return zero;
}
```

Может кому пригодится

Ответить

Ответы

Анонимный 11 марта 2013 г., 23:09

RSA прекрасно реализуется без длинной арифметики

slipstak2 11 марта 2013 г., 23:12

ссылка из википедии:
"Выбираются два различных случайных простых числа p и q заданного размера (например, 1024 бита каждое)."

Ответить

qwiqi 4 ноября 2013 г., 14:44

а почему не работает рисунок(?

Ответить

Анонимный 10 мая 2014 г., 11:05

Пожалуйста почините рисунок.
И еще, могли бы вы показать адаптацию алгоритма для osn = 10

Ответить

Ответы

slipstak2 10 мая 2014 г., 20:44

рисунок за 2 года пока не починили(. Надеюсь исправимся.
Что касается деления на 10. Не совсем понятно что вас интересует. вы можете смело исправлять osn на 10 и получите валидную реализацию деления длинного на длинное, но по osn = 10

Ответить

Unknown 4 апреля 2017 г., 22:04

деление 678 / 4 дает не то что надо. В чем проблема?

Ответить

Ответы

Unknown 10 апреля 2017 г., 00:07

Так что там? Решение то не верное.

Danil 10 апреля 2017 г., 01:05

Я бы не был так уверен, что решение неверное - скорее всего, проблема на вашей стороне.

Unknown 10 апреля 2017 г., 01:29

Dan Sagunov как на моей? Я делю 678 на 4 и получаю 1010. Если

делитель состоит из одной цифры то ответы не верные в большинстве случаев...

slipstak2 10 апреля 2017 г., 10:05

Ну во-первых, чтобы делать утверждения об ошибочной реализации, наверное, нужно показать исходный код, с которого вы запускаетесь.
Во-вторых, данный цикл статей не преследовал задачи предоставления целостной реализации под копипаст. Все-таки в некоторых вещах нужно поразбираться. Скорее всего у вас ошибка при вводе/инициализации/выводе.
В-третьих, ваше поведение считаю непрофессиональным. Если вы новичок и сидите на диване в ожидании готового решения, то это не есть путь война. Вам никто, ничего не должен! Если же вы бывалый и действительно нашли ошибку, то могли уже на нее указать, получив от нас благодарный фидбек.
У авторов блога нет задачи искать ошибки в чужих реализациях, даже если они основаны на фрагментах наших исходников. Но чтобы закрыть тему, для вас сделаем исключение

Unknown 11 апреля 2017 г., 21:46

Ок)
http://paste.ubuntu.com/24362452/ .cpp
http://paste.ubuntu.com/24362465/ .h

slipstak2 11 апреля 2017 г., 22:35

Ошибка в конструкторе по умолчанию: http://ideone.com/bpDQII

Ответить

Unknown 9 апреля 2017 г., 17:27

По коду:
1) curValue не инициализирован, но домножается на osn в методе LevelUp - как выглядит конструктор по умолчанию для BigInt?
2) Не очень понятна логика работы алгоритма в месте, где LevelUp делается только один раз. В таком случае лидирующие нули получаются, но мы лишний раз выполняем бинарный поиск для их получения. Почему бы сначала не определить минимальное число curValue, которое >= делителя?

Ответить

Ответы

Danil 9 апреля 2017 г., 17:41

1) Конструктор по умолчанию BigInt инициализирует его так, чтобы он соответствовал значению 0.
2) Уверен, что как раз логика вам понятна, поскольку вы написали, что мы лишь осуществим лишний проход бинпоиском. Вы правы - в случае, если curValue слишком мал, бинпоиск можно не выполнять.

Unknown 10 апреля 2017 г., 22:44

Спасибо!

Ответить

Чтобы оставить комментарий, нажмите кнопку ниже и войдите с аккаунтом Blogger.

ВОЙТИ С АККАУНТОМ BLOGGER

Следующее Главная страница Предыдущее

Подписаться на: Комментарии к сообщению (Atom)